# Segurança Semântica Não Implica Seletividade: Compilação de Regras Snort 3 para Switches Programáveis P4

Arthur de Freitas Medeiros
*Universidade Federal do Pampa (UNIPAMPA)*
Brazil
arthurfreitas.aluno@unipampa.edu.br

Silvio E. Quincozes
*Faculty of Computer Science (FACOM)*
*Federal University of Uberlândia*
Monte Carmelo, Brazil
sequincozes@ufu.br

***Abstract*—Programmable data planes can evaluate lightweight packet predicates before a software intrusion detection system (IDS), but Snort 3 rules also depend on payload, stream, flow, and service semantics that a P4 target cannot represent. We present a semantics-aware, target-aware Snort-to-P4 compiler that classifies the relation between each rule and its generated predicate as EXACT, NECESSARY, or UNSUPPORTED, records every omitted component in a compilation manifest, and fails open for packets outside an explicitly enforced IPv4 domain. Among 4,017 evaluated rules, 4,003 were classified as necessary, 14 as unsupported, and none as exact; the backend produced 331 P4 entries installed on BMv2. The global binary prefilter nevertheless marked 100% of parser-eligible packets as candidates in three CICIDS2017 traces. Observed alerts were preserved, and an end-to-end campaign over 1,000 sampled rules, with Snort 3 as the oracle and BMv2 as the target, found no violation. Retained provenance reduced the mean per-packet candidate rule set by 91–96% within the 3,611-rule P4-eligible partition, or 81.81–86.40% over the full ruleset. We also show, from the Snort 3.12.2.0 source, that for 2,931 service-based rules the necessary relation holds only where Snort evaluates the full header: once the flow's service is identified, header ports and protocol are not checked. Compiled with that semantics, the candidate-set reduction is bounded at 18.40% of the ruleset and measured at 18.29–18.30%. Semantic safety and operational selectivity are distinct properties, and what a rule-derived predicate guarantees depends on how the IDS evaluates the rule.**

## I. INTRODUÇÃO

**Contexto.** Este trabalho trata da tradução automática de regras de sistemas de detecção de intrusão (IDS) baseados em assinatura para switches programáveis. O IDS adotado é o Snort 3 [1], [2], cujas regras combinam condições de cabeçalho (protocolo, endereços, portas, flags TCP) com condições que dependem do conteúdo do pacote, do estado do fluxo e do serviço de aplicação. A linguagem alvo é P4 [3], que descreve o processamento de pacotes no plano de dados e executa em velocidade de linha, mas oferece um modelo computacional restrito: casamento por tabelas sobre campos de cabeçalho, sem inspeção de payload arbitrária e sem estado por fluxo no modelo usado. A ideia explorada é a pré-filtragem: deixar o switch decidir cedo quais pacotes precisam da inspeção completa do IDS em software.

**Motivação.** A inspeção em software é o gargalo reconhecido dessa arquitetura, e a literatura já demonstrou que o plano de dados programável pode aliviá-la. P4ID [4] deriva filtros diretamente de regras Snort e reporta redução no volume encaminhado ao IDS. O 4MIDable [5] reporta reduções no tráfego processado sem alterar as taxas de detecção observadas, em alvos BMv2 e Tofino. O P4-ONIDS [6] trata explicitamente das restrições de recursos do plano de dados por meio de agregação e deduplicação de regras. O conjunto desses resultados estabelece que a abordagem é viável e que há ganho a obter.

**Justificativa.** A fraqueza comum a esses trabalhos não está na viabilidade, e sim no que acontece com a parte da regra que não cabe no alvo. Uma regra Snort típica conjuga uma condição de cabeçalho com uma condição de payload ou de estado. Ao gerar o predicado P4, a condição que não cabe precisa de tratamento explícito, e, no escopo revisado, nenhum dos trabalhos analisados documenta, regra a regra, qual relação semântica sobrevive à tradução. Isso importa porque o erro é silencioso e assimétrico: se o predicado gerado ficar mais *restritivo* que a regra de origem, o pré-filtro descarta pacotes que a regra original detectaria, e a falha não aparece em nenhuma métrica de cobertura de tradução. Há ainda uma fonte de erro menos óbvia, que este trabalho documenta: o cabeçalho de uma regra não diz, sozinho, o que o IDS exige. No Snort 3, uma regra associada a um serviço de aplicação deixa de ter porta e protocolo checados quando o serviço do fluxo é identificado, e um predicado que os exija fica mais restritivo que a regra.

**Objetivo.** Este trabalho propõe um compilador Snort-para-P4 que torna explícitas as consequências semânticas da compilação parcial. A propriedade central que ele busca preservar é

$$\mathrm{Snort}(r, p) \Rightarrow \mathrm{P4_candidato}(r, p),$$

ou seja: se a regra $r$ dispararia sobre o pacote $p$, o predicado gerado obrigatoriamente marca $p$ como candidato. O predicado pode ser mais permissivo que a regra, nunca mais restritivo. Quando essa garantia não pode ser assegurada, a regra é rotulada como não suportada em vez de ser traduzida de forma aproximada, e pacotes que o parser do switch não

consegue interpretar são mantidos como candidatos (*fail-open*). As contribuições específicas são:

- Classificar a relação entre cada regra Snort e o predicado P4 gerado em três categorias — EXACT, NECESSARY e UNSUPPORTED — segundo a garantia efetivamente preservada;
- Registrar em um manifesto de compilação, predicado a predicado, o que foi preservado, delegado ao Snort, expandido ou tratado por fallback, mantendo a proveniência entrada-P4 $\rightarrow$ regras de origem através do lowering e da deduplicação;
- Implementar o backend completo, da regra até a instalação das entradas via P4Runtime em um alvo BMv2/v1model, com um domínio de pacotes imposto no plano de dados e validado por uma suíte controlada e pela contabilidade do replay;
- Avaliar separadamente a segurança semântica e a seletividade operacional, mostrando empiricamente que são propriedades independentes;
- Executar uma campanha diferencial fim a fim sobre 1.000 regras amostradas, com o Snort 3 como oráculo e o BMv2 como alvo;
- Mostrar, a partir do código-fonte do Snort 3.12.2.0, que regras *service-based* não tornam porta e protocolo do cabeçalho condições necessárias, medindo as 2.931 regras afetadas, quais planos continuam completos para elas e o que muda numa compilação fiel a essa semântica;
- Explorar um uso alternativo da proveniência retida — em vez de uma decisão binária, derivar o conjunto de regras candidatas por pacote — e medir os limites dessa redução.

## II. FUNDAMENTAÇÃO TEÓRICA

**Snort 3 e seu modelo de regras.** O Snort 3 é um IDS de rede baseado em assinaturas. Cada regra tem um cabeçalho — ação, protocolo, endereço e porta de origem, direção, endereço e porta de destino — e um corpo de opções entre parênteses. As opções incluem tanto condições de detecção (`content`, `pcre`, `flow`, `flowbits`, `flags`) quanto metadados (`msg`, `sid`, `classtype`). A distinção relevante aqui é que só uma parte dessas condições é avaliável sobre campos de cabeçalho; o restante exige inspeção de payload ou estado acumulado de conexões anteriores.

**Regras por porta e por serviço.** O Snort 3 distingue regras *port-based* de regras *service-based*. Uma regra é service-based quando declara a opção `service`, usa uma opção específica de um serviço ou nomeia um serviço no cabeçalho, como em `alert http` [2]. Numa regra port-based, todo componente do cabeçalho precisa casar. Numa regra service-based, uma vez que o Snort identifica o serviço do fluxo como um dos serviços da regra, os endereços continuam sendo checados, mas as portas e o protocolo não (Seção V-C). Um predicado de cabeçalho extraído da regra é, portanto, uma aproximação cuja relação com a regra precisa ser declarada, e não presumida.

**P4 e o modelo v1model.** P4 [7] é uma linguagem para descrever o processamento de pacotes no plano de dados. O programa define o parser de cabeçalhos, tabelas de casamento e ações. O modelo arquitetural usado aqui é o v1model. Uma tabela P4 casa campos extraídos do cabeçalho segundo tipos de casamento (`exact`, `lpm`, `ternary`, `range`) e dispara uma ação. O modelo não oferece varredura de payload nem estado por fluxo, o que estabelece o limite do que pode ser traduzido.

**BMv2 e P4Runtime.** O BMv2 [8] é o switch de referência em software usado como alvo de execução. As entradas das tabelas não fazem parte do programa P4: são instaladas em tempo de execução pelo plano de controle via P4Runtime [9], um protocolo gRPC que usa o arquivo P4Info gerado pelo compilador P4 para identificar tabelas, campos e ações por identificadores numéricos.

**Pré-filtragem e predicados necessários.** Um pré-filtro é uma condição avaliada antes do IDS que decide se o pacote precisa de inspeção. Diz-se que o predicado $p_r$ é *necessário* para a regra $r$ quando todo pacote que satisfaz $r$ também satisfaz $p_r$. Um predicado necessário pode ser usado com segurança para pré-filtragem, porque nunca descarta um pacote que a regra detectaria — ainda que retenha tráfego adicional. É essa noção que fundamenta a propriedade preservada neste trabalho.

**CICIDS2017.** Conjunto de traços de tráfego produzido pelo Canadian Institute for Cybersecurity [10], com janelas rotuladas por tipo de ataque. As janelas de quinta-feira (SQL Injection, XSS e Brute Force) foram usadas na avaliação.

## III. TRABALHOS RELACIONADOS

**P4ID [4].** Deriva filtros de pacotes diretamente de regras Snort, usando endereços IP, portas de transporte, protocolos e flags TCP, e combina estágios de processamento sem estado e com estado. Sua principal contribuição é demonstrar que filtragem derivada de regras é viável no plano de dados programável e que o encaminhamento ao IDS pode ser reduzido. A limitação central, do ponto de vista deste trabalho, é que a extração de campos não vem acompanhada, no escopo revisado, da declaração da relação semântica entre a regra de origem e o filtro gerado: não fica registrado o que foi descartado ao converter uma regra que também dependia de payload.

**P4-ONIDS [6].** Trata explicitamente das limitações de recursos de planos de dados restritos, aplicando representação de faixas, agregação de regras, deduplicação e uso de informação de fluxo para limitar os pacotes encaminhados ao NIDS. A contribuição é mostrar que a viabilidade em hardware real depende de compressão da representação. A limitação é que a agregação e a deduplicação são tratadas como otimizações de ocupação de tabela, sem que o efeito dessas transformações sobre a garantia semântica de cada regra agregada seja explicitado.

**4MIDable [5].** Propõe um arcabouço para integrar middleboxes de segurança existentes a uma infraestrutura P4/SDN. Seu agente P4ID suporta processamento com estado e realimentação para um IDS não modificado, e foi avaliado tanto em BMv2 quanto em Tofino, reportando redução de tráfego processado sem afetar as taxas de detecção observadas. A contribuição é a integração prática com IDS legado e a validação em hardware. A limitação é que a preservação da detecção é estabelecida

empiricamente sobre os traços avaliados, e não por uma relação por regra que valha independentemente do tráfego observado.

**P4NIPS [11].** Apresenta uma implementação recente de prevenção de intrusão em P4, estendendo a linha de detecção para ação de bloqueio no plano de dados. Contribui com uma realização concreta e atual do conceito. A limitação, para os fins deste trabalho, é a mesma das anteriores: o foco está na construção do sistema, não na caracterização do que a tradução preserva ou perde.

**eRBF — Brum et al. [12].** Revisita a pré-filtragem baseada em regras considerando que o IDS pode precisar de tráfego que não satisfaz diretamente as condições extraídas da regra. A abordagem incorpora contexto de fluxo, incluindo estabelecimento e encerramento de conexões TCP e dependências de protocolo selecionadas, e é avaliada com Snort 3 sobre conjuntos grandes de regras. A contribuição é mostrar que pré-filtragem agressiva precisa levar em conta o contexto exigido pelo IDS — um argumento diretamente alinhado ao deste trabalho. A limitação é que a avaliação é feita em software sobre traços, sem tratar do mapeamento dessas semânticas para as restrições de processamento e de recursos de um alvo P4.

**RNA [13].** Segue direção complementar, orientada a eventos: analisa eventos selecionados de scripts Zeek e os mapeia para templates de protocolo e de evento para dispositivos de encaminhamento programáveis, junto a mecanismos no host. A contribuição é demonstrar a geração coordenada de funcionalidade distribuída entre host e plano de dados. A limitação, quanto ao escopo aqui tratado, é que parte de eventos Zeek selecionados e de templates predefinidos, e não de uma classificação semântica geral de regras de IDS por assinatura segundo a garantia preservada.

**Contexto mais amplo.** Surveys sobre fundamentos e aplicações de P4 [14], sobre aplicações de segurança em switches P4 [15], sobre IDS em plano de dados programável [16] e trabalhos orientados à implementação [17] documentam a amplitude do campo e reforçam que o mapeamento de funcionalidade de segurança para alvos programáveis é limitado por recursos e por expressividade.

### A. Posicionamento

A Tabela I compara os sistemas quanto à origem das regras, ao alvo avaliado, à existência de uma relação semântica declarada e à proveniência por regra. O padrão que emerge é consistente: os trabalhos demonstram viabilidade, integração e eficiência de representação, mas tratam a tradução parcial como um detalhe de implementação. Em nenhum deles, no escopo revisado, é explícita, por regra, a garantia que o predicado gerado preserva, nem a rastreabilidade da entrada instalada de volta às regras que a originaram. É essa lacuna que este trabalho endereça.

Tabela I
COMPARAÇÃO COM OS TRABALHOS RELACIONADOS. N.E.: NÃO EXPLÍCITO NO ESCOPO REVISADO, O QUE NÃO EQUIVALE A AUSÊNCIA DO RECURSO.

| Trabalho | Origem | Alvo | Semântica declarada | Proveniência por regra |
|---|---|---|---|---|
| P4ID | Snort | Plano P4 | N.E. | N.E. |
| P4-ONIDS | NIDS | Plano P4 | N.E. | N.E. |
| 4MIDable | Agente P4ID | BMv2, Tofino | N.E. | N.E. |
| P4NIPS | N.E. | P4 | N.E. | N.E. |
| eRBF | Snort 3 | Software | N.E. | N.E. |
| RNA | Zeek | P4 + host | N.E. | N.E. |
| **Este trabalho** | **Snort 3** | **BMv2** | **Sim (3 classes)** | **Sim** |

## IV. PROPOSTA

A proposta é um compilador que traduz regras Snort 3 em entradas de tabela P4 buscando preservar a propriedade $\mathrm{Snort}(r,p) \Rightarrow \mathrm{P4_candidato}(r,p)$ e tornando explícito tudo aquilo que não pôde ser traduzido. O pipeline tem nove estágios:

1) **Parser** — lê a sintaxe da regra Snort 3 e produz uma representação estruturada, rejeitando as formas que o próprio Snort rejeitaria;
2) **Resolução de variáveis** — substitui variáveis de rede (`$HOME_NET`, `$EXTERNAL_NET`) e de serviço pelos valores do perfil de configuração;
3) **IR semântica** — converte a regra em uma lista de predicados tipados (cabeçalho, payload, estado de fluxo, metadados);
4) **Análise de capacidade do alvo** — para cada predicado, decide se ele é representável na tabela P4 e atribui um tratamento: PRESERVED, EXPANDED, DELEGATED, FALLBACK ou OUT_OF_SCOPE;
5) **Classificador semântico** — a partir dos tratamentos, classifica a regra como EXACT (todo o significado cabe no P4), NECESSARY (o predicado é mais permissivo, e seguro) ou UNSUPPORTED (não há predicado seguro derivável);
6) **Manifesto** — registra a decisão de cada predicado de cada regra, com o motivo, e mantém a ligação entrada $\rightarrow$ regras de origem;
7) **Backend P4** — gera as entradas, expande listas de portas e regras bidirecionais, normaliza e deduplica;
8) **Lowering** — converte para valores e máscaras ternárias;
9) **Instalação** — gera o plano P4Runtime e instala no BMv2, verificando o contrato do P4Info e conferindo a codificação por round-trip.

O ponto de projeto que diferencia a proposta é o tratamento do caso em que a tradução não é possível. Em vez de descartar a condição que não cabe — o que tornaria o predicado mais restritivo e abriria um buraco de detecção —, a condição é delegada ao Snort e registrada no manifesto.

**Domínio de pacotes imposto.** A garantia pressupõe que o plano de dados leia os campos a que o predicado se refere: um parser que assumisse cabeçalho IPv4 fixo de 20 bytes leria bytes de opções como portas. O programa P4 define por isso um domínio suportado $D$: Ethernet sem VLAN com IPv4 direto, IHL $= 5$, sem fragmentação (flags nulas ou DF) e sem erro de parser. O parser só extrai a camada de transporte dentro de $D$, e o controle de ingresso repete a checagem antes de aplicar a tabela. Cada pacote incrementa exatamente um

de três contadores: `P4_CANDIDATE` (alguma entrada casou dentro de $D$), `P4_NO_MATCH` (nenhuma casou dentro de $D$) e `PACKET_DOMAIN_FALLBACK` (fora de $D$, marcado como candidato incondicionalmente). O veredito *no-match*, o único que poderia justificar retirar um pacote do IDS, só é emitido dentro de $D$.

**Semântica de cabeçalho de regras service-based.** O compilador tem dois modos. O modo `legacy`, padrão e usado em todos os artefatos congelados, avalia o cabeçalho de uma regra service-based como o de uma regra por porta e registra o serviço como condição delegada. O modo `snort3` delega também porta e protocolo dessas regras, mantendo o protocolo apenas quando a opção `flags` já as restringe a TCP. A Seção V-C mede a diferença entre os dois.

## V. Avaliação

A avaliação responde a quatro questões: (Q1) qual a cobertura da compilação semântica; (Q2) se o backend gerado é viável no alvo; (Q3) qual a seletividade do pré-filtro resultante; e (Q4) se os alertas observados são preservados e se a proveniência retida tem utilidade quando a seletividade binária é baixa. As três primeiras caracterizam a proposta; a última verifica empiricamente a garantia semântica, que a Seção V-C confronta ainda com o modo como o Snort 3 avalia o cabeçalho.

### A. Metodologia

**Conjunto de regras e perfil.** O conjunto avaliado é o Snort 3 Community Ruleset, com 4.017 regras, sob o perfil experimental `cicids2017`, que resolve `$HOME_NET` para 192.168.10.0/24 e `$HTTP_PORTS` para 80 e 8080. As classificações dependem dessa configuração, e o perfil resolvido é registrado junto aos resultados.

**Tráfego.** Janelas de quinta-feira do CICIDS2017: SQL Injection (35.274 quadros) e XSS (191.848) como traços de desenvolvimento, e Brute Force (1.108.658) mantido como traço *held-out* — nenhum ajuste de política foi feito após inspecioná-lo.

**Ambiente.** Os alertas de referência vêm do Snort 3.12.2.0. O programa P4 é compilado pelo `p4c` 1.2.4.2, com o backend `p4c-bm2-ss`, e executado no BMv2 (`simple_switch_grpc` 1.15.0, modelo v1model) em Ubuntu 22.04, com as entradas instaladas via P4Runtime. O replay usa um socket AF_PACKET que transmite os bytes Ethernet capturados sem reserialização, após instalação nova do pipeline, contadores conferidos em zero e uma auditoria de ociosidade com delta nulo; os quadros enviados são reconciliados com a soma dos três contadores.

**Validação do domínio.** Uma suíte controlada envia ao BMv2, com o plano de 331 entradas instalado, um quadro por caso e compara o delta dos contadores com a expectativa offline. Os onze casos cobrem candidatos TCP, UDP e ICMP, um no-match garantido, flags TCP exatas e mascaradas, opções IPv4, primeiro fragmento com MF, fragmento não inicial, cabeçalho IPv4 truncado e cabeçalho TCP truncado.

**Campanha diferencial fim a fim.** Os traços só exercitam as poucas regras que disparam neles. Para testar a garantia sobre mais regras, um gerador sintetiza, a partir da representação intermediária de cada regra, um fluxo curto cuja testemunha deveria fazer o Snort disparar aquela regra: handshake TCP quando `flow` exige sessão estabelecida, cabeçalho com os valores da regra e cada `content` no buffer inspecionado. Cada testemunha tem um vizinho sem o conteúdo (negativo de payload) e, quando as entradas instaladas deixam espaço, um com porta de destino fora de todas elas (negativo de cabeçalho). Foi possível sintetizar testemunha para 2.142 das 4.017 regras; das 1.875 excluídas, 1.654 têm opções como `pcre`, `byte_test` ou `flowbits`, que não se satisfazem escolhendo bytes. Uma amostra de 1.000 regras, estratificada por protocolo, serviço e direção com semente fixa, gerou 6.684 quadros, todos dentro de $D$. O Snort lê o traço com o ruleset completo, o BMv2 recebe os quadros um a um, e cada caso recebe um veredito: CONFIRMADO (o Snort alertou e todo quadro alertado foi candidato); VIOLAÇÃO (o Snort alertou e a testemunha, ou outro quadro que satisfaz o cabeçalho da regra, foi no-match — o único resultado que contradiz a propriedade); VÁCUO (o Snort não alertou, e o caso não testou nada); ou DEPENDÊNCIA DE ESTADO (a testemunha foi candidata, mas o Snort atribuiu o alerta a um quadro da sessão que não satisfaz o cabeçalho e foi no-match). A campanha rodou contra o plano global e contra o plano do discriminador forte (Seção V-B).

**Métricas.** (i) cobertura de classificação semântica; (ii) número de entradas instaladas, sua validação e a redução do estado de instalação, $1 - N_{\text{install}}/N_{\text{source}}$; (iii) taxa de pacotes candidatos, calculada apenas sobre $D$; (iv) preservação de alertas, comparando a execução do Snort com o ruleset completo contra a união das execuções com a partição candidata e com a partição UNSUPPORTED; e (v) conjunto de regras candidatas por pacote, $C(x) = \bigcup_{e \in E(x)} S(e)$, em que $E(x)$ são as entradas que casam o pacote $x$ e $S(e)$ as regras de origem da entrada $e$. Sua redução é $R(x) = 1 - |C(x)|/B$ dentro da partição P4-elegível de $B$ regras e $R_{\text{full}}(x) = 1 - (|C(x)| + F)/4017$ sobre o ruleset, em que $F$ conta as regras de fallback, que valem para todo pacote.

**Reprodutibilidade.** O código, o perfil, os scripts de análise e os artefatos congelados estão em https://github.com/medeiros08/Snort-p4. Os dez artefatos do pipeline (manifesto semântico, partição de regras, entradas P4 e plano P4Runtime) são determinísticos: regenerados em uma cópia isolada a partir do código versionado, saíram idênticos byte a byte aos congelados. A suíte automatizada tem 23 módulos de teste; 20 passam e os outros 3, que exigem os bindings P4Runtime/gRPC, só executam no ambiente com o BMv2.

### B. Resultados

**Cobertura semântica (Q1).** Das 4.017 regras, 4.003 foram classificadas como NECESSARY, 14 como UNSUPPORTED e **nenhuma** como EXACT. O parser leu 4.017 de 4.017 regras sem falha. No nível de predicado: 21.300 preservados, 20.621 delegados ao Snort, 2.662 expandidos, 82 em fallback e **zero** não avaliados — isto é, não houve predicado sobre o qual o compilador deixasse de decidir. O fato de EXACT ser zero é o resultado esperado, e não um defeito: nenhuma regra do

Tabela II
SELETIVIDADE E PRESERVAÇÃO POR TRAÇO. CANDIDATOS SOBRE OS PACOTES EM $D$; A COLUNA DO DISCRIMINADOR FORTE FOI CALCULADA COM O CLASSIFICADOR OFFLINE. ALERTAS DO SNORT COM O RULESET COMPLETO; P4 CAND.: PACOTES DE ALERTA MARCADOS COMO CANDIDATOS.

| Traço | Em $D$ | Global cand. | Forte cand. | Alertas | SIDs | P4 cand. |
|---|---|---|---|---|---|---|
| SQLi | 34.924 | 100% | 99,015% | 150 | 1 | 150 |
| XSS | 189.076 | 100% | 98,869% | 528 | 2 | 528 |
| Brute Force | 1.101.459 | 100% | 98,942% | 2.391 | 5 | 2.391 |

Tabela III
CAMPANHA DIFERENCIAL SOBRE A MESMA AMOSTRA DE 1.000 REGRAS. "SÓ PRÓPRIA": CONFIRMADAS CUJA TESTEMUNHA CASOU APENAS ENTRADAS DA PRÓPRIA REGRA. A RODADA GLOBAL USOU UMA REVISÃO ANTERIOR DO GERADOR DE TESTEMUNHAS; AS REVISÕES MUDARAM SÓ A CONSTRUÇÃO DAS TESTEMUNHAS E REDUZIRAM OS CASOS VÁCUOS.

| Plano | Entr. | Conf. | Viol. | Dep. estado | Fallb. oper. | Vácuo | Só própria |
|---|---|---|---|---|---|---|---|
| Global | 331 | 923 | 0 | 0 | – | 77 | 0 |
| Forte | 322 | 851 | 0 | 2 | 107 | 40 | 153 |

conjunto tem todo o seu significado representável no plano de dados, porque praticamente todas combinam cabeçalho com `content` ou com estado de fluxo. As 14 UNSUPPORTED não são ignoradas: quatro têm cabeçalho não seletivo, dez são regras de serviço sem cabeçalho tradicional, e todas permanecem no Snort. A Seção V-C mostra que, das 4.003 NECESSARY, a garantia vale integralmente para 1.072; para as outras 2.931, vale só enquanto o Snort avalia o cabeçalho inteiro.

**Viabilidade do backend (Q2).** As 6.675 instâncias de predicado geradas, expandidas e normalizadas resultaram em 327 casamentos únicos e, após o lowering de flags TCP, em 331 entradas de instalação — redução de 95,04% no *estado da tabela*, não no tráfego, obtida por deduplicação de casamentos idênticos. As 331 entradas foram codificadas e instaladas sem erro de codificação ou de validação, dentro da capacidade de 8.192 entradas da tabela, e a proveniência liga cada entrada às regras que a originaram.

**Domínio de pacotes.** A suíte controlada passou nos onze casos com deltas exatos; os cinco fragmentados ou truncados caíram em `PACKET_DOMAIN_FALLBACK`. Nos traços, ficaram fora de $D$ 350, 2.772 e 7.199 quadros, quase todos ARP e IPv6, e todos mantidos como candidatos. O replay reconcilia quadro a quadro com os contadores nos três traços; no Brute Force, a primeira rodada perdeu 87 de 1.108.658 quadros no transporte, e uma nova, com filas maiores, contou todos, igual ao classificador offline.

**Seletividade: o resultado negativo (Q3).** O pré-filtro binário global marcou **100% dos pacotes elegíveis** como candidatos nos três traços (Tabela II). A análise causal identificou entradas amplas: `p4-106` casa qualquer TCP e subsume 246 das 331 entradas, e `p4-40` e `p4-138` casam destino e origem em `HOME_NET`. A ablação com discriminador forte considera forte a entrada que restringe porta, flags TCP ou prefixo IPv4 diferente de exatamente `HOME_NET`, e mantém uma regra P4-elegível só se todas as suas entradas forem fortes. Ela removeu nove entradas, manteve 322 para 3.611 regras elegíveis e pôs 392 em fallback operacional, além das 14 UNSUPPORTED; mesmo assim, marcou 98,869–99,015% dos pacotes nos traços de desenvolvimento e 98,942% no held-out. A cobertura quase universal persiste, portanto, por acúmulo da união entre predicados necessários distribuídos, e não por umas poucas regras excessivamente amplas.

**Preservação de alertas (Q4).** Nos três traços, a partição semântica não produziu alertas faltantes nem adicionais, e todos os pacotes de alerta elegíveis foram candidatos (Tabela II). As colunas P4 foram medidas no BMv2 reproduzindo só os pacotes de alerta, de novo com o programa corrigido e com contabilidade exata; a reclassificação offline dá as mesmas contagens. No held-out, a auditoria do discriminador forte cobriu 2.289 eventos por predicados retidos e 102 por fallback, sem nenhum faltante. Registra-se, contudo, que nenhuma das 14 regras UNSUPPORTED disparou e que os traços acionaram apenas 1, 2 e 5 SIDs distintos, o que limita o poder de discriminação dessa verificação. Os nomes das janelas também não implicam que os alertas observados detectaram os ataques rotulados.

**Campanha fim a fim (Q4).** Nenhum caso resultou em VIOLAÇÃO em nenhum dos dois planos (Tabela III), nenhum quadro caiu fora de $D$, e as duas atribuições de alertas a casos, por faixa de pacotes e por timestamp, concordaram em todos. No plano global, 923 regras foram confirmadas e 77 foram vácuas; as 1.000 testemunhas e os 1.000 negativos de payload foram candidatos, e os 101 negativos de cabeçalho foram no-match. Nenhuma confirmação desse plano, porém, pode ser atribuída só ao predicado da própria regra: toda testemunha confirmada também casou uma entrada de outra regra. É a mesma cobertura ampla que elimina a seletividade, agora eliminando o poder de falsificar o lowering de uma regra isolada. No plano do discriminador forte, 851 regras foram confirmadas, 153 delas somente pelas próprias entradas, de modo que um defeito de lowering nessas regras teria aparecido como violação. Nas duas rodadas, a categoria prevista pelo casador offline coincidiu com o contador do BMv2 em todos os 2.101 quadros de testemunhas e vizinhos. Os dois casos de dependência de estado, SIDs 43809 e 47386, são regras de requisição HTTP cujo alerta o Snort atribui ao ACK do servidor que dispara a inspeção da requisição armazenada, e o plano forte marca esse ACK como no-match. O predicado vale para a testemunha, mas um sistema que retirasse do IDS os pacotes no-match poderia atrasar ou suprimir esses alertas.

**Redução do espaço de regras candidatas (Q4).** Usando a proveniência retida em vez da decisão binária, o conjunto médio de regras candidatas por pacote, dentro da partição P4-elegível de 3.611 regras, foi reduzido em 95,04% (SQLi), 96,11% (XSS) e 91,01% (Brute Force *held-out*, com média de 324,52 regras por pacote), como mostra a Tabela IV. As 406 regras de fallback (392 operacionais e 14 UNSUPPORTED) valem para todo pacote; contadas contra o ruleset completo,

Tabela IV
CONJUNTO DE REGRAS CANDIDATAS POR PACOTE (OFFLINE, PLANO DO DISCRIMINADOR FORTE). $R$ SOBRE AS 3.611 REGRAS P4-ELEGÍVEIS; $R_{\text{full}}$ SOBRE AS 4.017, CONTANDO AS 406 DE FALLBACK; PROT.: REDUÇÃO QUANDO SE ENCAMINHAM GRUPOS INTEIROS POR PROTOCOLO.

| Traço | Média | Mediana | Máx. | $R$ | $R_{\text{full}}$ | Prot. |
|---|---|---|---|---|---|---|
| SQLi | 179,28 | 27 | 1.228 | 95,04% | 85,43% | 19,10% |
| XSS | 140,49 | 13 | 1.228 | 96,11% | 86,40% | 15,65% |
| Brute Force | 324,52 | 241 | 1.228 | 91,01% | 81,81% | 12,30% |

as reduções são de 85,43%, 86,40% e 81,81%. A redução vem inteiramente da proveniência fina entrada → regra: um pré-filtro que encaminhasse grupos inteiros de regras perderia a maior parte dela. Agrupando por protocolo, com um grupo TCP de 3.439 regras, a redução cai para 12,30–19,10%; protocolo e direção chegam a 54–57%, protocolo e serviço a 81–92%, e só protocolo, direção e serviço, com 308 grupos para 322 entradas, fica a 0,01 ponto percentual do nível de entrada.

### C. Regras service-based e o limite da garantia

A premissa de que uma regra só dispara quando todo o seu cabeçalho casa vale para o Snort 3 apenas em parte. No Snort 3.12.2.0, antes de checar o cabeçalho de uma regra com serviços, o Snort compara o serviço identificado no fluxo com os da regra (`src/detection/detection_options.cc` e `src/detection/rtn_checks.cc`). Se o serviço é um deles, as checagens de porta são puladas, a de protocolo passa incondicionalmente, e só os endereços são checados; se é outro, a regra é rejeitada; só sem serviço identificado o cabeçalho inteiro é avaliado. Como a identificação do serviço pode vir da inspeção do conteúdo, uma requisição HTTP numa porta fora de `$HTTP_PORTS` ainda é avaliada pelas regras HTTP. A exceção são as regras que nomeiam o serviço no cabeçalho, como `alert http`, que o Snort marca como regras de usuário e que mantêm as portas checadas (`src/parser/parse_rule.cc`).

No modo `legacy`, o compilador trata o serviço como mais uma condição delegada e mantém porta e protocolo no predicado, o que seria correto se o Snort exigisse serviço *e* cabeçalho. Das 4.003 regras NECESSARY, 2.931 são afetadas: 2.928 declaram `service` (`http` em 2.314, `imap` em 150, `pop3` em 138, `smtp` em 131), e três (SIDs 529, 3238 e 3239) recebem `dcerpc` e `netbios-ssn` das opções `dce_iface` e `dce_stub_data` (`src/detection/fp_utils.cc`). Todas têm protocolo no predicado, e 2.877 também porta; para elas, $\text{Snort}(r,p) \Rightarrow \text{P4_candidato}(r,p)$ só vale para pacotes cujo serviço o Snort não identificou. A garantia vale integralmente para as outras 1.072, incluindo as 12 regras `alert http`. Um predicado restrito aos endereços seria necessário para todas as 2.931 e ainda seletivo para 2.924.

Se isso se traduz em candidato perdido depende do plano inteiro. No plano global, as entradas sem porta nem protocolo `p4-40` e `p4-138` contêm a região de alerta de 2.922 dessas regras, e `p4-106` contém a parte TCP das nove restantes (SNMP, TFTP, HTTP e Kerberos). O plano global é, portanto, completo para elas, exceto para pacotes não TCP dessas nove. O plano do discriminador forte remove justamente essas entradas e não cobre a região de nenhuma das 2.877 regras que pré-filtra — e é dele que derivam os conjuntos de regras candidatas da Tabela IV.

Recompilado no modo `snort3`, o ruleset dá 3.996 regras NECESSARY e 21 UNSUPPORTED, e o backend produz 235 entradas finais em vez de 331. A mesma política de discriminador forte mantém apenas 739 regras elegíveis, porque um predicado só de endereço sobre `HOME_NET` não é discriminador forte, e 3.278 regras passam a valer para todo pacote: nenhum conjunto candidato fica abaixo delas, o que limita a redução sobre o ruleset a 18,40%. Medida offline nos três traços, com o mesmo espelho do parser e o mesmo casador que reproduzem exatamente os números congelados no modo `legacy`, a compilação corrigida mantém o pré-filtro global em 100%; o discriminador forte marca 92,298%, 92,173% e 94,885% dos pacotes; o conjunto médio cai para 3,97, 3,94 e 4,48 das 739 regras elegíveis; e a redução sobre o ruleset fica em 18,30%, 18,30% e 18,29%. A auditoria dos 3.069 alertas não perde nenhum, mas só porque todos passam a vir de regras em fallback: a SID 254, responsável por 2.960 deles, declara `service:dns` e, sem UDP e porta 53, fica apenas com o destino `HOME_NET`. Os traços, portanto, não exercitam os conjuntos candidatos corrigidos. Nenhum plano da compilação corrigida foi instalado no BMv2. Um teste no BMv2 com 200 regras `service:http` na porta 18080 confirmou o achado: o Snort alertou em 190, e o plano forte marcou todas como no-match.

### D. Discussão dos resultados

Os achados centrais são três. O primeiro é negativo: segurança semântica não implica seletividade operacional, e a mesma união de predicados que elimina a seletividade também elimina o poder de teste — no plano global, nenhuma confirmação da campanha dependia só da regra testada. O segundo é construtivo, mas limitado: a proveniência indica quais regras importam para cada pacote, porém a redução vem da granularidade por entrada, desaparece com agrupamento grosseiro e repousa em portas que o Snort 3 não usa depois de identificar o serviço. Um seletor de candidatas que usasse também o serviço identificado — que o próprio Snort usa para rejeitar regras de outros serviços — é o passo natural seguinte e não foi avaliado aqui. O terceiro é que a garantia de um predicado derivado de regra depende de como o IDS avalia a regra, e não só do texto do cabeçalho: no plano global, as entradas amplas que eliminaram a seletividade foram as mesmas que o mantiveram completo para as regras service-based. Todas as medidas de conjunto candidato são de viabilidade *offline*: não foi implementada execução do Snort orientada por conjunto de candidatos.

### E. Limitações

Os resultados valem para um ruleset, um perfil de variáveis e um conjunto de dados antigo em relação ao tráfego atual. Para as quantidades por pacote, medimos também, offline,

a quinta-feira inteira (9.322.025 quadros, 9.240.405 em $D$): o pré-filtro global continua em 100%, o discriminador forte marca 97,534%, e a redução média é de 93,73% na partição elegível, 84,25% sobre o ruleset e 18,29% com a compilação corrigida; as janelas não são atípicas do dia, mas os alertas do Snort não foram calculados para o restante dele. A campanha amplia a cobertura de regras para 851–923, mas com tráfego sintético, um ponto por regra e sempre numa porta do próprio cabeçalho, de modo que não observa a avaliação service-based; as 1.875 regras sem testemunha sintetizável não foram testadas. A semântica service-based foi estabelecida pela leitura do código-fonte e da documentação do Snort 3.12.2.0 e confirmada no BMv2 só para um serviço e uma porta. O alvo é o BMv2 em software: não houve medição de vazão, latência ou recursos de hardware. Por fim, algumas condições de cabeçalho representáveis ainda são omitidas, o que só alarga os predicados: faixas de porta, tipo e código ICMP e `ip_proto`. Representá-las elevaria a redução sobre o ruleset do plano forte em no máximo 1,8, 3,1 e 0,07 ponto percentual, respectivamente.

## VI. Conclusão e Trabalhos Futuros

Sistemas de detecção de intrusão baseados em assinatura enfrentam custo crescente de inspeção em software, e a pré-filtragem em switches programáveis é a resposta natural. A literatura demonstrou a viabilidade dessa abordagem, mas tratou a tradução parcial de regras como detalhe de implementação: quando uma condição de payload ou de estado não cabe no plano de dados, o que foi feito com ela não é explicitado. Isso é um problema mal resolvido porque o erro resultante é silencioso — um predicado mais restritivo que a regra de origem descarta tráfego que deveria ser inspecionado, sem que nenhuma métrica de cobertura de tradução o revele.

Este trabalho desenvolveu um compilador Snort-para-P4 que busca garantir que todo pacote sobre o qual uma regra dispararia seja marcado como candidato, e que registra explicitamente cada componente semântico omitido. Das 4.017 regras avaliadas, 4.003 resultaram em predicados necessários e 14 foram declaradas não suportadas; o backend instalou 331 entradas em um alvo BMv2, com fail-open fora do domínio que o parser interpreta. Os alertas observados foram preservados, e a campanha fim a fim sobre 1.000 regras não encontrou violação. O principal resultado é negativo e informativo: o pré-filtro binário marcou 100% dos pacotes como candidatos. A proveniência retida reduz em 91 a 96% o conjunto de regras candidatas por pacote, mas essa redução depende de portas que o Snort 3 ignora em 2.931 regras service-based; com a semântica do Snort 3, fica limitada a 18,4% do ruleset. A contribuição para a literatura é a separação explícita entre segurança semântica e seletividade operacional, acompanhada de um manifesto de compilação que torna auditável o que cada tradução preserva, e a constatação de que essa garantia depende de como o IDS avalia a regra, e não apenas do seu texto.

Como trabalhos futuros, quatro direções se destacam. A primeira é medir no BMv2 os planos da compilação corrigida. A segunda é um seletor de regras candidatas que use o serviço identificado, além do cabeçalho, e a execução do Snort orientada por esse conjunto, transformando a redução medida *offline* em ganho efetivo — o que exige tratar as dependências de estado entre regras, como as das duas regras HTTP da campanha. A terceira é ampliar a expressividade do backend, incorporando faixas de porta por decomposição ternária e campos ICMP à chave da tabela. A quarta é avaliar o custo de execução em alvos de hardware e sobre conjuntos de regras, perfis e tráfegos mais amplos, incluindo medição de vazão e latência, que não foram objeto desta avaliação.

## Referências


[1] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*. Seattle, WA, USA: USENIX Association, 1999, pp. 229–238.

[2] The Snort Team, *Snort 3 User Manual*, 2026, snort 3.12.2.0, https://github.com/snort3/snort3.

[3] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[4] B. Lewis, M. Broadbent, and N. Race, "P4ID: P4 enhanced intrusion detection," in *2019 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2019.

[5] B. Lewis, M. Broadbent, C. Rotsos, and N. Race, "4MIDable: Flexible network offloading for security VNFs," *Journal of Network and Systems Management*, vol. 31, p. 52, 2023.

[6] K. Tavares and T. Ferreto, "P4-ONIDS: A P4-based NIDS optimized for constrained programmable data planes in SDN," in *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, 2021, pp. 434–447.

[7] The P4 Language Consortium, *$P4_{16}$ Language Specification*, 2023, version 1.2.4, https://p4.org/specs/.

[8] p4lang, "BMv2: The behavioral model software switch," https://github.com/p4lang/behavioral-model, 2025, `simple_switch_grpc` 1.15.0, commit 68f4a978.

[9] The P4.org API Working Group, *P4Runtime Specification*, 2024, version 1.4.1, https://p4.org/specs/.

[10] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[11] E. V. Senoski, J. R. Andreotii, R. K. Machnicki, J. P. Correia, and V. Fulber-Garcia, "P4NIPS: P4 network intrusion prevention system," in *Anais Estendidos do Simpósio Brasileiro de Cibersegurança (SBSeg)*, 2025, pp. 75–84.

[12] H. B. Brum, L. A. D. Knob, T. Ferreto, and D. Siracusa, "Rethinking NIDS rule-based pre-filtering," in *IFIP/IEEE International Symposium on Integrated Network Management and Network Operations (CNSM)*, 2025.

[13] L. S. Hagen, A. S. Ilha, R. Parizotto, and L. P. Gaspary, "RNA: Automating IDS/IPS event detection offload into programmable forwarding devices," in *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2025.

[14] F. Hauser, M. Häberle, D. Merling, S. Lindner, V. Gurevich, F. Zeiger, R. Frank, and M. Menth, "A survey on data plane programming with P4: Fundamentals, advances, and applied research," *Journal of Network and Computer Applications*, vol. 212, p. 103561, 2023.

[15] A. AlSabeh, J. Khoury, E. Kfoury, J. Crichigno, and E. Bou-Harb, "A survey on security applications of P4 programmable switches and a STRIDE-based vulnerability assessment," *Computer Networks*, vol. 207, p. 108800, 2022.

[16] M.-R. Fida and A. H. Ahmed, "Leveraging programmable data plane for network intrusion detection: A survey," *Computer Networks*, vol. 281, p. 112202, 2026.

[17] A. Mazloum, A. AlSabeh, E. Kfoury, and J. Crichigno, "Security applications in P4: Implementation and lessons learned," *Computer Networks*, vol. 257, p. 111011, 2025.